

AN-NAJAH NATIONAL UNIVERSITY

Vehicle Rental Management System

Final Project Report - Phase 1 and Phase 2

Course	Software Engineering Project
Team Members	Mohammad abu salah - 12324714 Fadi Dallal - 12340833
Repository	https://github.com/najah-vehicle-rental/Vehicle-Rental-Management-System
Analysis Platform	SonarQube Cloud + GitHub Actions
Submission Date	1 August 2026

Executive Summary

This report documents the implementation, continuous-integration setup, static analysis, refactoring, testing, code-coverage improvement, Java documentation, and UML design of the Vehicle Rental Management System. The project uses Java, Maven, JUnit 5, Mockito, JaCoCo, GitHub Actions, and SonarQube Cloud. The final analysis passed the Quality Gate, reduced the open issue count from 152 to 147, and increased overall coverage from 78.3% to 86.8%.

Repository	https://github.com/najah-vehicle-rental/Vehicle-Rental-Management-System
Main branch	main
Working approach	Feature branches + pull requests + automated checks
Final coverage	86.8% overall; 89.4% line; 80.5% condition
Final Quality Gate	Passed
Generated documentation	javaDoc/index.html
UML sources	docs/VehicleRentalSystem.puml and docs/VehicleRentalSystem.svg

1. Project Overview

The system supports manager authentication, vehicle catalog browsing, rentals, double-booking prevention, rental-duration validation, reminders, returns, billing, late penalties, and type-specific rules for cars, vans, trucks, motorcycles, and electric vehicles.

1.1 Architecture

The application follows a layered architecture: Presentation, Application/Service, Domain, and Persistence. Strategy is used for vehicle-specific rental validation, Observer is used for rental event notifications, and Factory centralizes creation of vehicle subtypes.

- Presentation: console interaction in Main.
- Service: authentication, rental, return, billing, reminder, notification, and vehicle services.
- Domain: Vehicle hierarchy, Rental, Manager, and RentalRequirements.
- Persistence: text-file repositories for managers, vehicles, and rentals.
- Patterns: Strategy, Observer, and Factory.

2. Tooling and Continuous Integration

The Maven build was connected to SonarQube Cloud through a GitHub Actions workflow. The workflow checks out the repository, configures Java, runs tests and JaCoCo, and publishes the analysis. Pull requests were reviewed only after automated checks completed successfully.

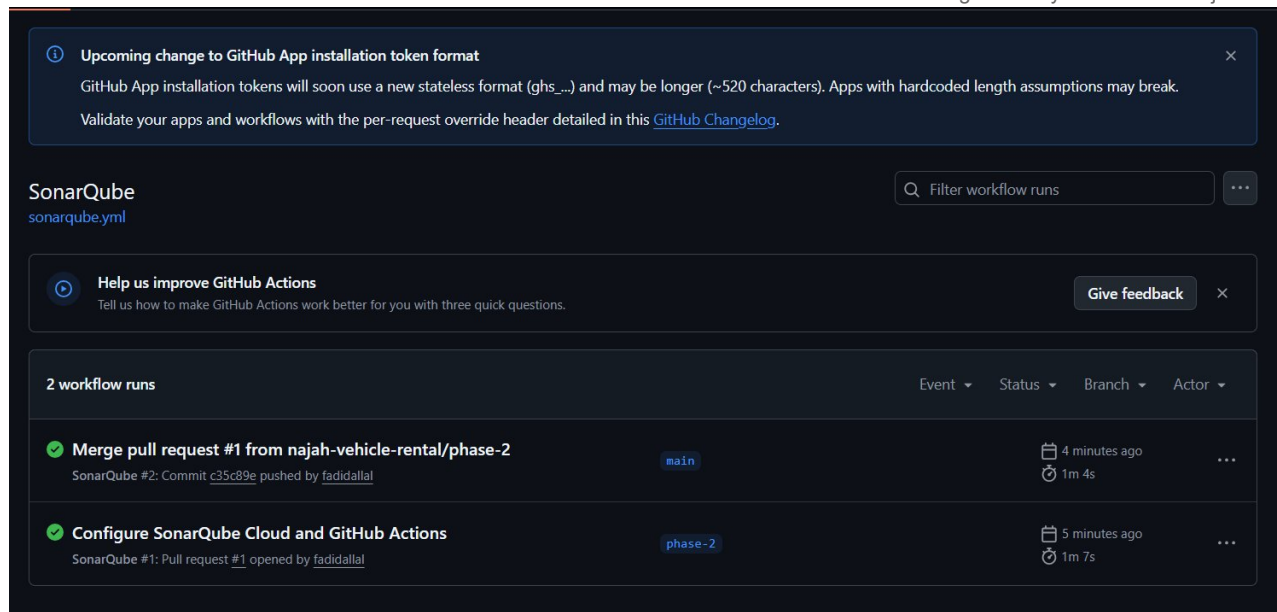


Figure 1. Successful GitHub Actions workflow runs for SonarQube analysis.

3. Static Analysis Before Refactoring

The initial SonarQube Cloud analysis reported 152 open issues, 0.0% duplication, and 78.3% overall coverage. Although the Quality Gate passed, the issue list contained maintainability problems that were suitable for focused refactoring.

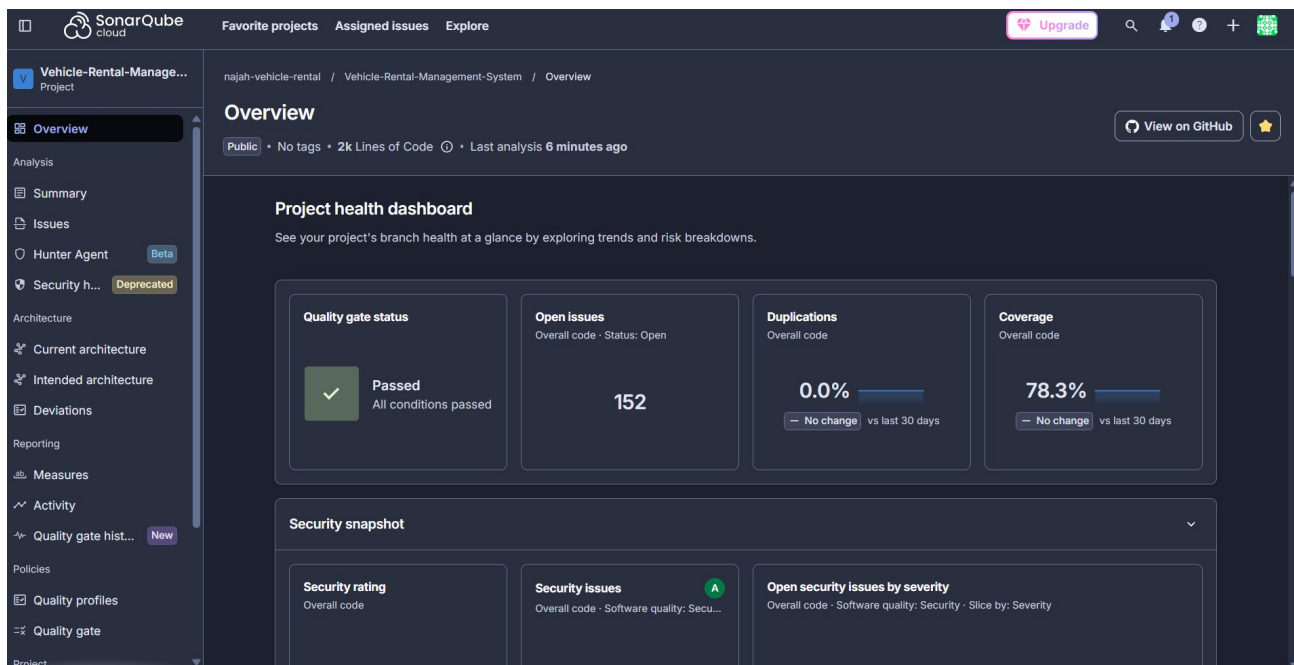


Figure 2. Initial SonarQube Cloud overview: 152 issues and 78.3% coverage.

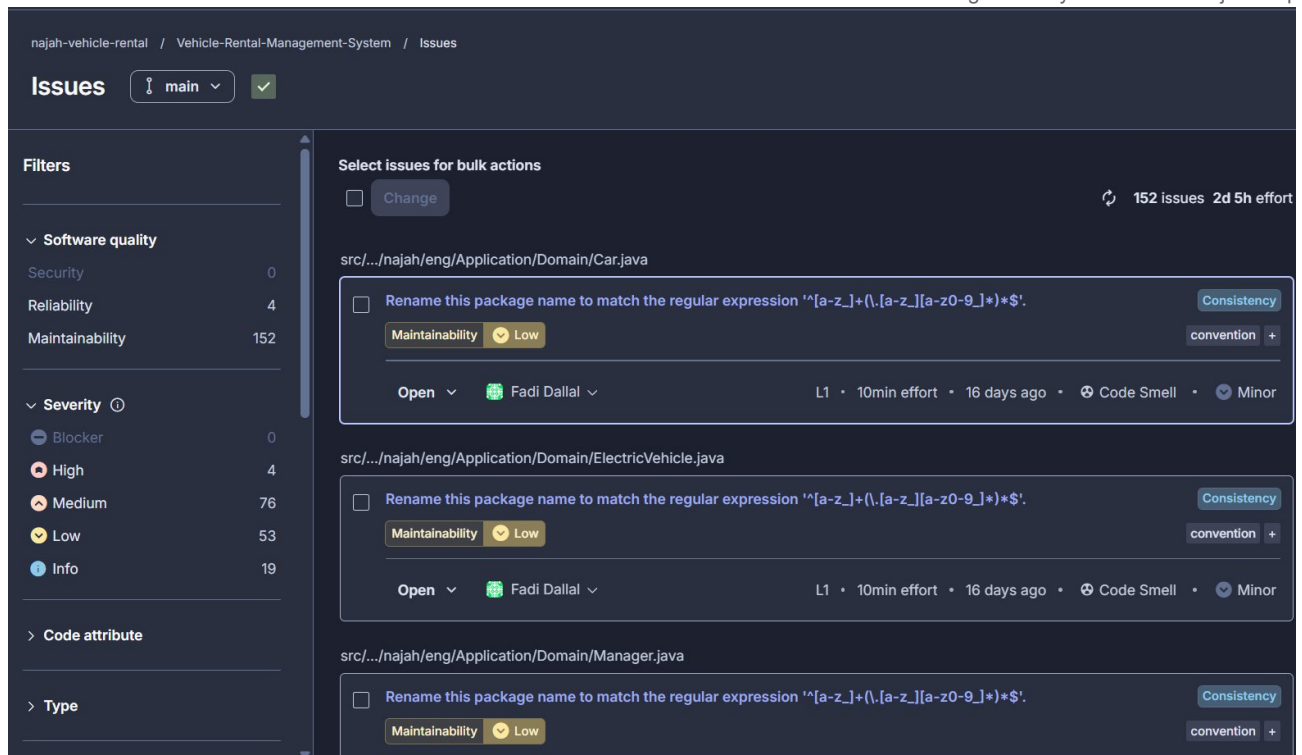


Figure 3. Initial issue distribution before the selected refactorings.

4. Selected Code Smells and Refactoring

Three different code smells were selected. Each change was intentionally small and behavior-preserving, and each was validated using the existing and expanded test suite.

4.1 Duplicated String Literal in VehicleRepository

SonarQube reported that the literal "Vehicles file not found." was repeated three times. Repeated messages are harder to maintain because a future wording change must be applied in multiple locations.

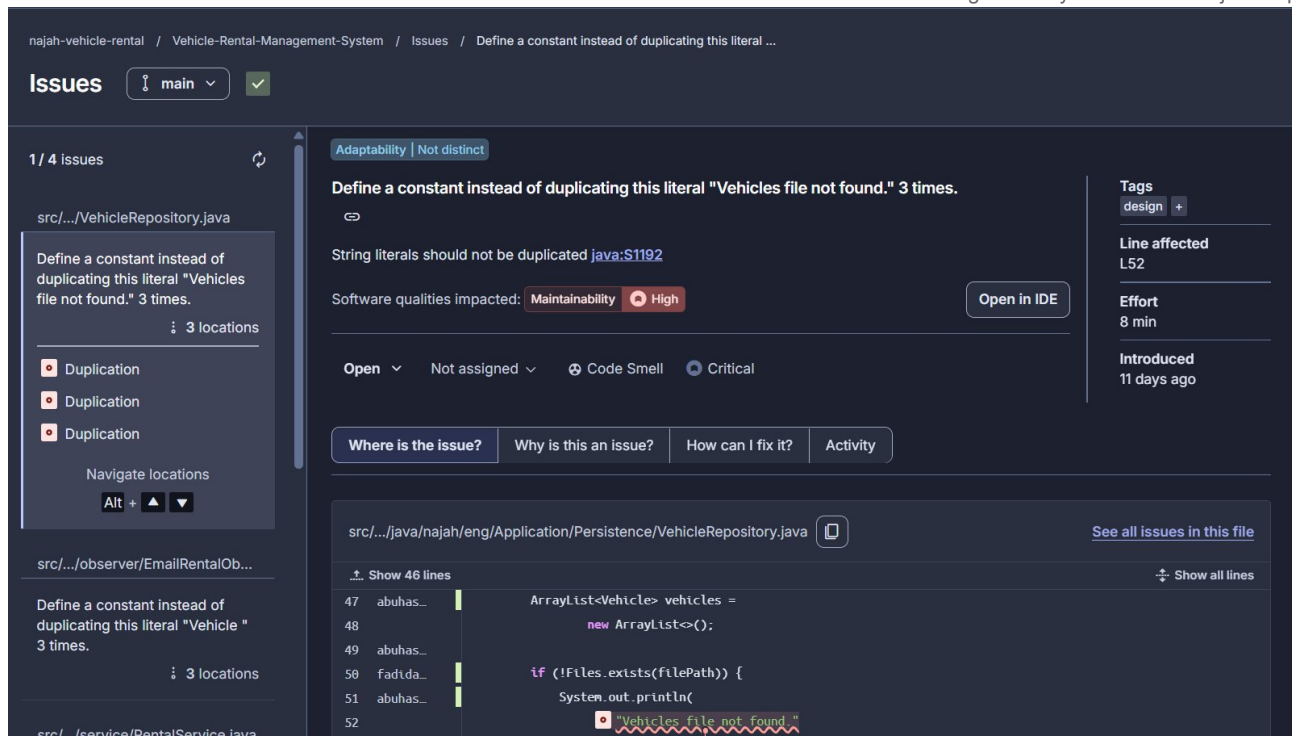


Figure 4. SonarQube issue for the duplicated vehicle-file message.

Before

```
if (!Files.exists(filePath)) {
    System.out.println("Vehicles file not found.");
    return vehicles;
}
```

After

```
private static final String VEHICLES_FILE_NOT_FOUND =
    "Vehicles file not found.";

if (!Files.exists(filePath)) {
    System.out.println(VEHICLES_FILE_NOT_FOUND);
    return vehicles;
}
```

Refactoring approach: Extract Constant. The message now has a single source of truth without changing repository behavior.

4.2 Test File Path Did Not Match the Package

ReminderServiceTest declared package najah.eng.Application.service, but its directory used dots as part of one directory name. This violated the expected Java package-to-directory mapping.

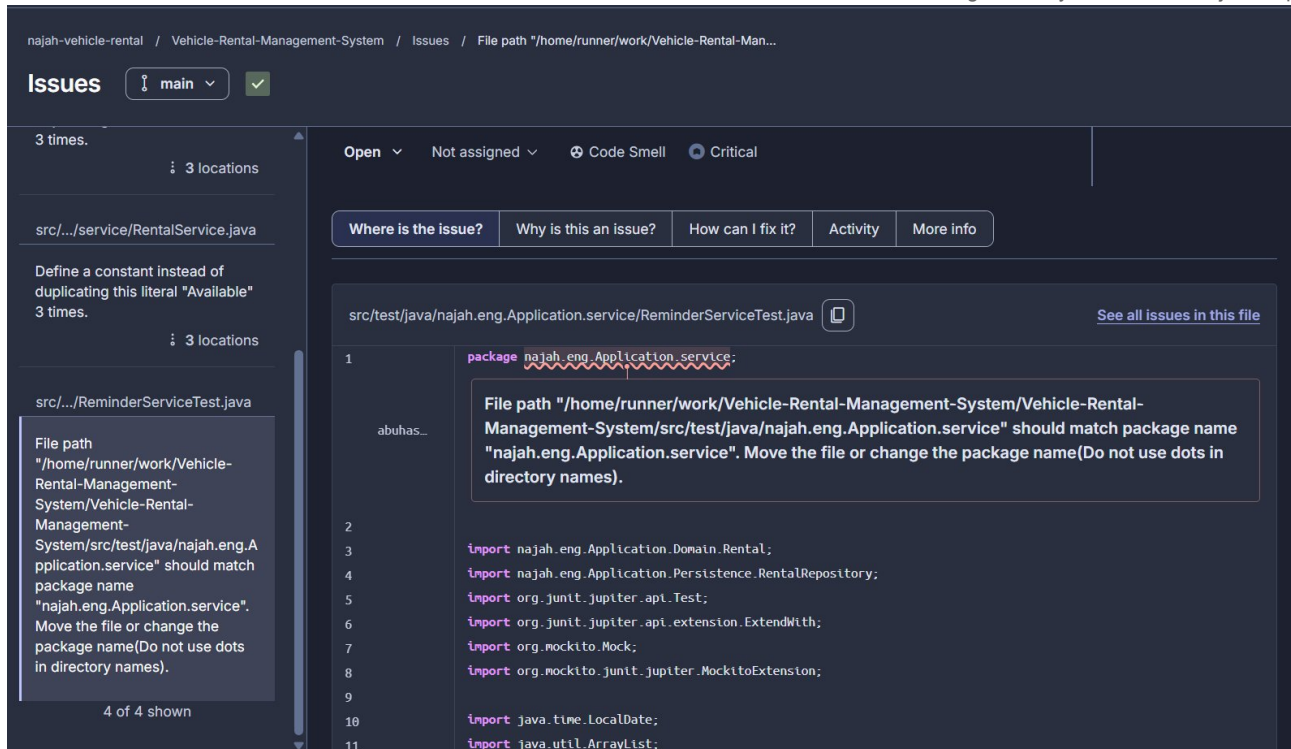


Figure 5. SonarQube issue for the mismatched test file path.

Before:

```
src/test/java/najah.eng.Application.service/ReminderServiceTest.java
```

After:

```
src/test/java/najah/eng/Application/service/ReminderServiceTest.java
```

Package declaration remained:

```
package najah.eng.Application.service;
```

Refactoring approach: Move File. No test logic or package declaration was changed; only the directory structure was corrected.

4.3 Direct System.out Logging in RentalRepository

RentalRepository printed exception messages directly to standard output. Repository-level failure reporting is more maintainable with a logger because it supports severity, stack traces, and configurable handlers.

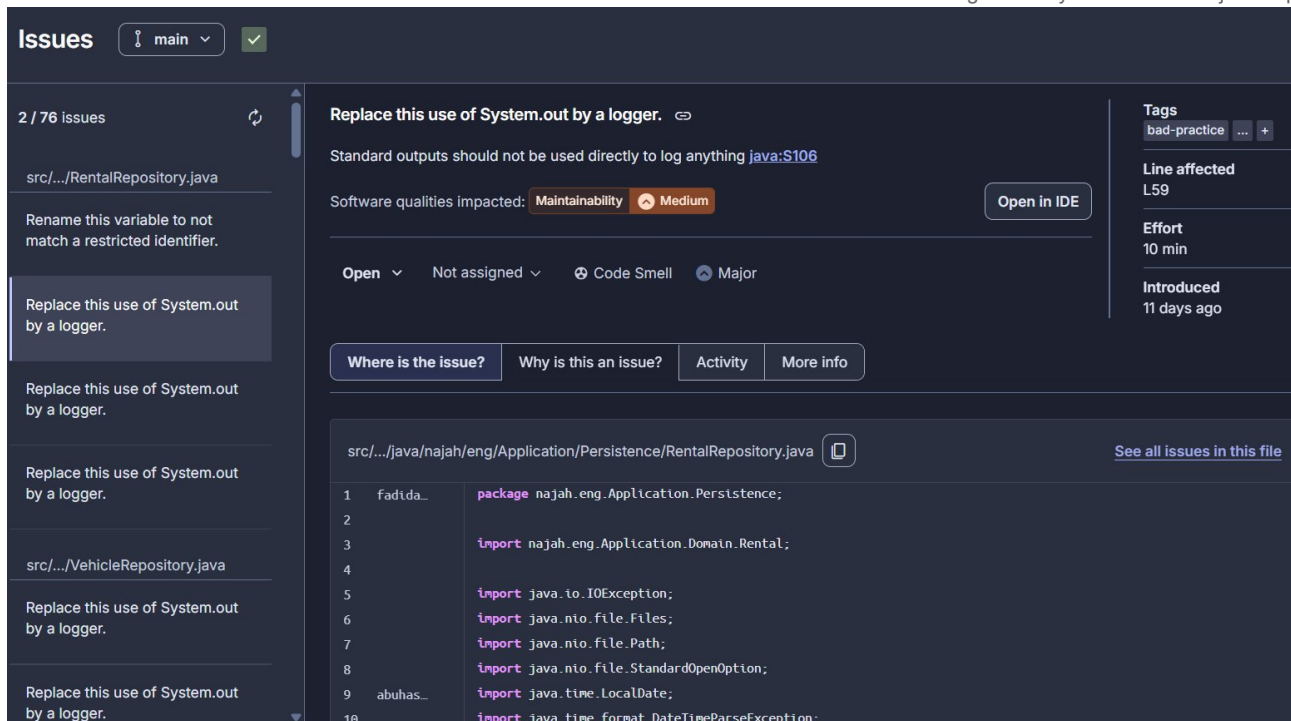


Figure 6. SonarQube issue recommending a logger instead of System.out.

Before

```
catch (IOException e) {
    System.out.println("Error saving rental.");
    return false;
}
```

After

```
private static final Logger LOGGER =
    Logger.getLogger(RentalRepository.class.getName());

catch (IOException e) {
    LOGGER.log(Level.SEVERE, "Error saving rental.", e);
    return false;
}
```

Refactoring approach: Replace Console Output with Logger. Equivalent changes were applied to save, read, and close-rental error paths. Error-path tests were added to force IOException conditions and confirm that each operation returns the expected result.

5. Testing and Coverage Improvement

Coverage was improved through targeted tests instead of changing production behavior. RentalRepository error cases were exercised using temporary directory paths that force IOException. EmailNotificationService output was captured and verified. RentalService tests were expanded to cover validation boundaries, unavailable and missing vehicles, active-rental conflicts, repository failures, rollback behavior, and vehicle-specific rules.

Metric	Before	After	Result
Overall coverage	78.3%	86.8%	+8.5 percentage points
Open issues	152	147	5 issues removed

High-severity issues	4	2	Reduced by 50%
Medium-severity issues	76	73	3 fewer
Duplications	0.0%	0.0%	No regression
Final line coverage	-	89.4%	62 uncovered lines
Final condition coverage	-	80.5%	46 uncovered conditions

Conditions to Cover	0
Uncovered Conditions	0
Overall Code	
Coverage	86.8%
Lines to Cover	583
Uncovered Lines	62
Line Coverage	89.4%
Conditions to Cover	236
Uncovered Conditions	46
Condition Coverage	80.5%
Tests	

Figure 7. Final SonarQube coverage metrics.

6. Results After Refactoring

After the selected refactorings, SonarQube reported 147 open issues instead of 152. High-severity issues decreased from 4 to 2 and medium-severity issues decreased from 76 to 73. The project continued to report 0.0% duplication. The pull-request analysis reported no new issues, 100% coverage on new code, and a passed Quality Gate.

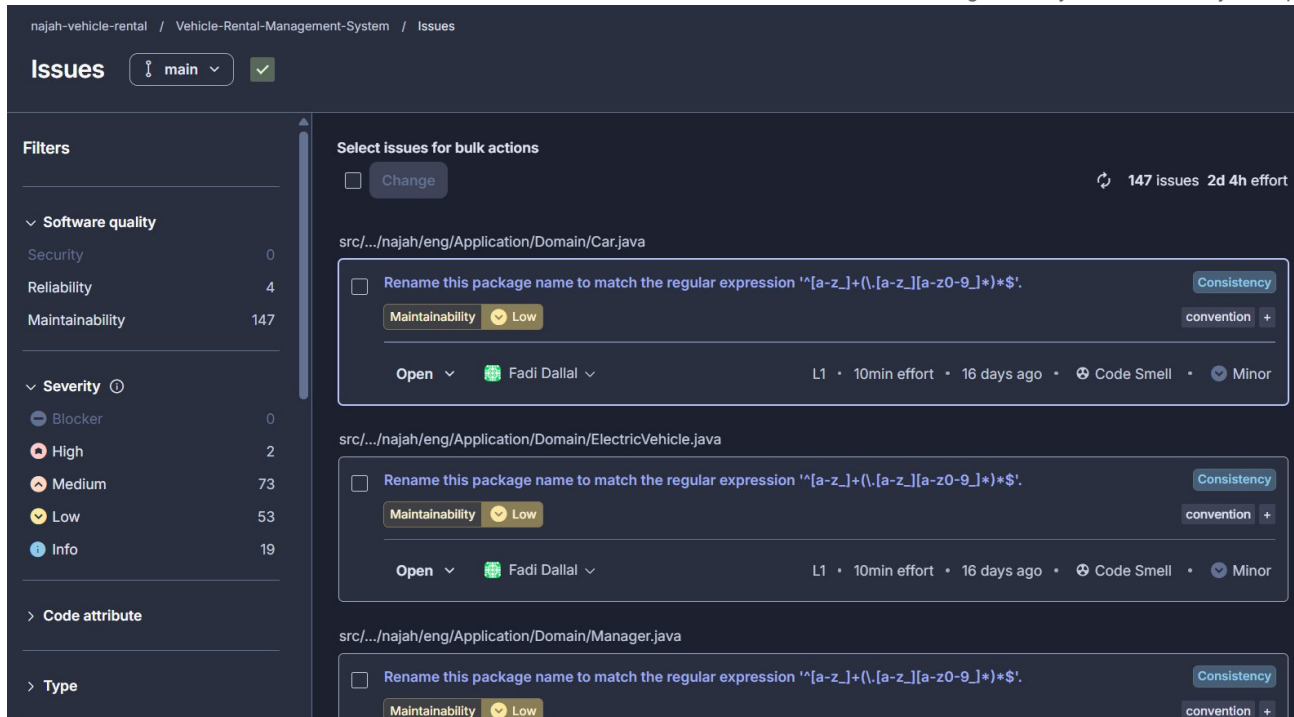


Figure 8. Issue count after the selected refactorings: 147 open issues.

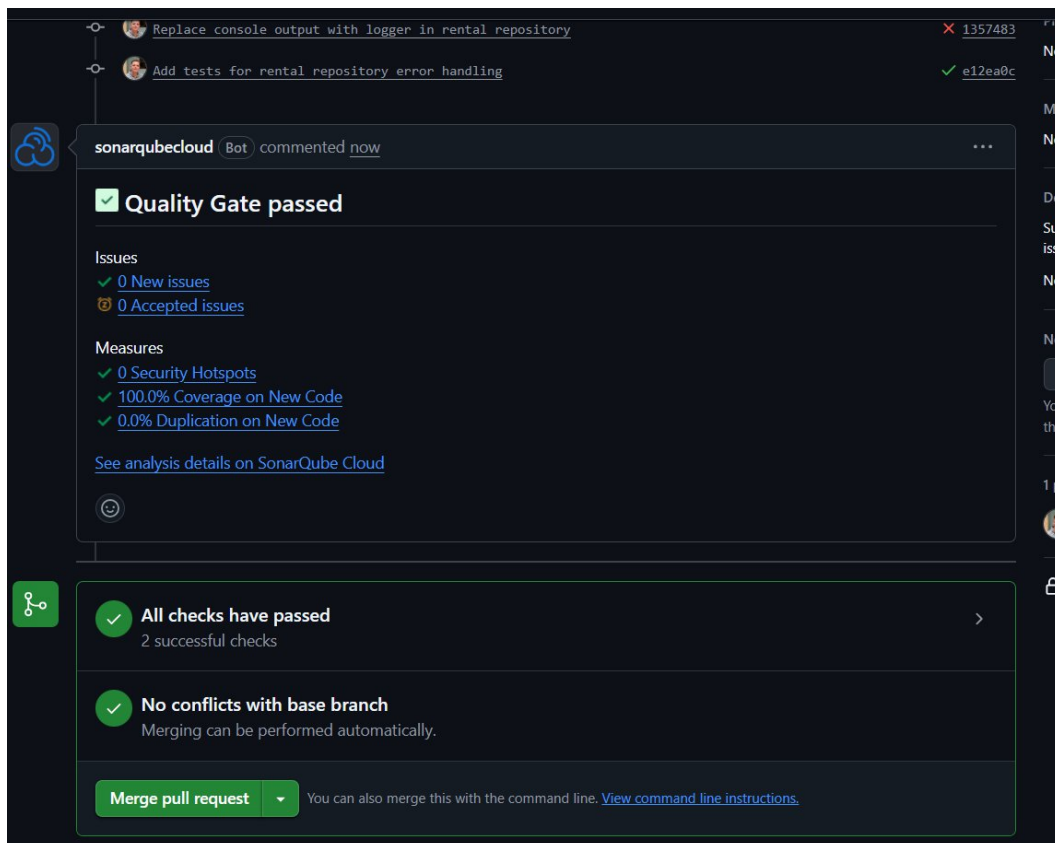


Figure 9. Pull-request Quality Gate passed with no new issues.

7. Java Documentation

The Maven Javadoc configuration originally generated documentation for only Vehicle, RentalService, and ReturnService. The restrictive sourceFileIncludes block was removed and the visibility setting was changed

to private so that documentation is generated for all production classes and members. The final HTML entry point is `javaDoc/index.html`.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-javadoc-plugin</artifactId>
  <version>3.12.0</version>
  <configuration>
    <show>private</show>
    <encoding>UTF-8</encoding>
    <docencoding>UTF-8</docencoding>
    <charset>UTF-8</charset>
    <failOnWarnings>false</failOnWarnings>
    <outputDirectory>${project.build.directory}/javadoc</outputDirectory>
  </configuration>
</plugin>
```

The generated documentation includes the Domain, Persistence, factory, observer, presentation, service, and strategy packages.

8. UML Class Diagram

The complete UML diagram was generated from the final design and stored as PlantUML and SVG. The diagram shows the vehicle inheritance hierarchy, service-to-repository dependencies, the Strategy interfaces and implementations, the Observer publisher and observers, and supporting domain classes.

Vehicle Rental Management System - Complete UML

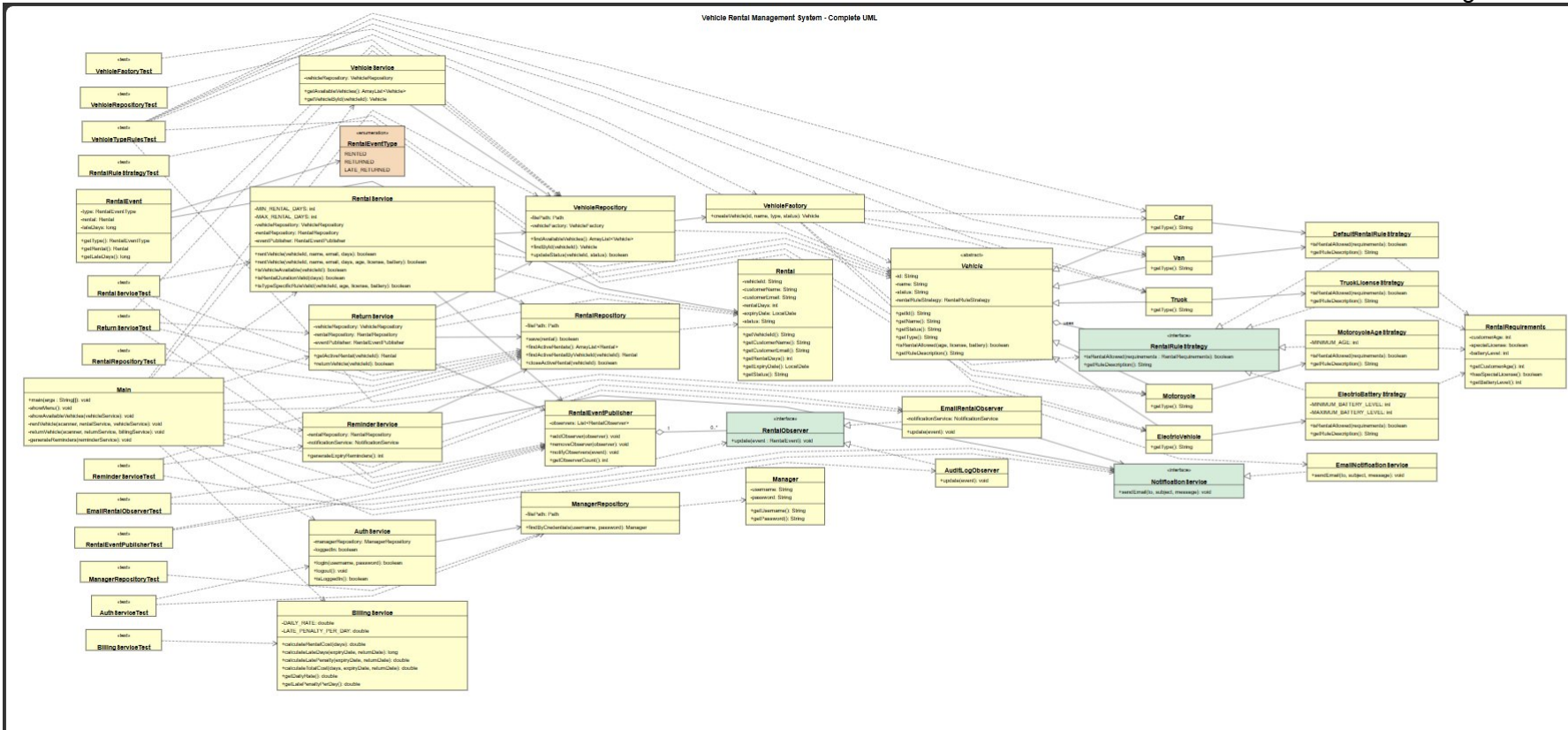


Figure 10. Complete UML class diagram.

9. Final Verification and Submission Package

- Source code and public GitHub repository link.
- GitHub Actions workflow and successful SonarQube Cloud analysis.
- Before-and-after SonarQube evidence.
- Three code smells with refactoring explanations and code examples.
- Overall coverage above 80% (final result: 86.8%).
- Generated Java documentation in javaDoc/.
- Complete UML diagram in PlantUML and SVG formats.
- Separate AI Refactoring Report.
- Final Maven clean and verify run with BUILD SUCCESS before submission.

10. Conclusion

The project satisfies the main implementation and quality objectives. The selected refactorings improved maintainability without changing functional behavior, automated tests protected the changes, and the final coverage exceeded the required threshold. The project also includes continuous analysis, generated Java documentation, and a complete UML model.

Appendix A - Evidence Files

- Initial overview: image(266).png
- Initial issues: image(267).png
- Duplicated literal issue: image(268).png
- Package/path issue: image(269).png
- System.out logger issue: image(271).png
- Quality Gate result: image(276).png
- After issues: image(277).png
- Final coverage: image(282).png
- UML diagram: docs/VehicleRentalSystem.svg